

Reporte técnico de diseño y validación del módulo iot de
monitoreo y control de energía

VoltGuard

Proyecto: EDSIA 2026

Equipo:

Jafet de los Angeles Palacios Guatzozón

Juliana Hernández

Luis khaled Reyes Casanova

Tabla de contenido

1. Introducción.....	3
2. Objetivo.....	4
3. Justificación.....	4
4. Materiales y Métodos	8
4. Metodología	22
4.1. Diseño de Módulo	22
4.2. Arquitectura general de la plataforma	28
4.3. API expuesta	32
4.4. Seguridad: autenticación, cifrado y los dos niveles de corte.....	34
4.5. Comunicación en tiempo real: WebSocket sobre REST existente	35
4.6. Inteligencia del sistema: predicción de consumo y detección de anomalías..	35
4.7. El dashboard web (front/).....	36
4.8. La aplicación móvil (mobile/)	38
4.9. Despliegue: contenedores, proxy inverso y TLS.....	38
4.10. Calidad: pruebas automatizadas e integración continua	38
5. Resultados	39
5.1. Casos de uso — Administración de la plataforma	39
5.2. Casos de uso — Usuario final	43
6. Conclusiones	46
7. Trabajo futuro.....	49

1. Introducción

En este proyecto desarrollamos un módulo electrónico IoT destinado al monitoreo y control de una carga conectada a la red eléctrica doméstica de aproximadamente 127 VCA. El objetivo general consiste en disponer de un dispositivo capaz de medir las principales variables eléctricas de una carga, transmitir posteriormente dicha información mediante Wi-Fi hacia un servidor, almacenar históricos de consumo y permitir tanto el control remoto como la desconexión automática ante condiciones de consumo consideradas anormales.

Desde el comienzo planteamos que el dispositivo no debía limitarse a funcionar como un interruptor conectado a Internet. El sistema debía integrar tres funciones principales:

1. Medición eléctrica.
2. Control físico de alimentación de la carga.
3. Comunicación IoT y registro histórico.

Adicionalmente, consideramos desde el diseño la incorporación de una aplicación móvil desde la cual podamos consultar el estado de cada dispositivo, observar consumo energético, consultar históricos y ordenar el encendido o apagado de la carga.

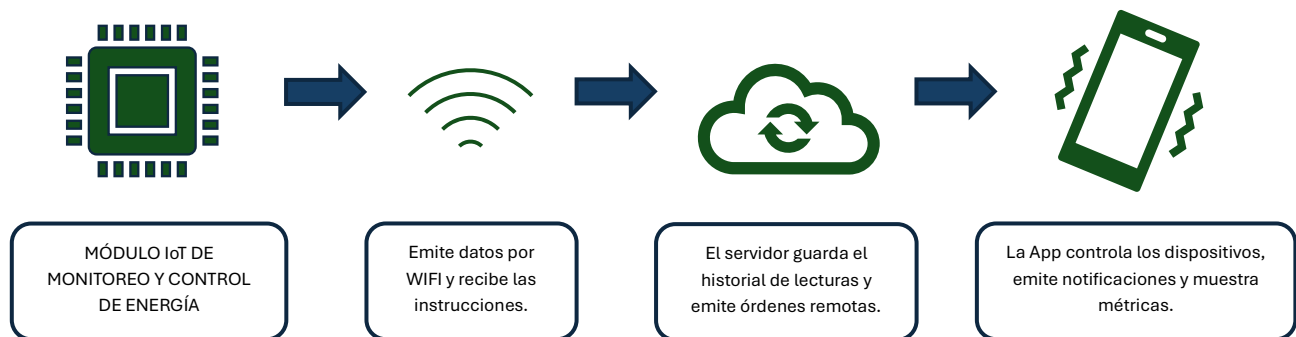
De esta idea general surge VoltGuard. Se trata de una plataforma de monitoreo, control y protección eléctrica construida alrededor de un concepto de "breaker inteligente": un dispositivo físico que se instala en línea con una carga eléctrica (un electrodoméstico, un circuito de una casa o un taller), mide su consumo en tiempo real y puede cortar la corriente automáticamente ante una condición de riesgo (sin depender de que haya conexión a Internet en ese momento) o manualmente, a petición de la persona usuaria desde una aplicación. El objetivo del proyecto no fue únicamente demostrar que la parte electrónica, si no, construir todo lo que rodea a ese hardware para que deje de ser un experimento de laboratorio y se convierta en un sistema utilizable: una identidad y credenciales propias para cada dispositivo, un servidor central que sepa la verdad del sistema, aplicaciones reales para que una persona lo use desde el celular o desde una computadora, y una infraestructura de despliegue que permita instalarlo en una máquina real sin depender de servicios de terceros.

2. Objetivo

Diseñar un módulo IoT conectado a la red eléctrica con capacidad para:

- medir tensión eficaz de la red;
- medir corriente consumida por la carga;
- calcular o adquirir potencia activa;
- conocer factor de potencia;
- medir frecuencia de red;
- registrar energía acumulada;
- conectar y desconectar físicamente la carga;
- incorporar posteriormente protección por sobreconsumo;
- conectarse mediante Wi-Fi;
- transmitir telemetría a un servidor;
- almacenar información histórica;
- recibir comandos remotos;
- integrarse posteriormente con una aplicación móvil.

La arquitectura general prevista es:



3. Justificación

VoltGuard representa una solución de valor porque integra en una sola plataforma elementos que normalmente se encuentran aislados: instrumentación eléctrica, sistemas embebidos, conectividad IoT, backend, persistencia de datos,

control remoto, aplicaciones de usuario, administración, análisis de consumo y mecanismos de seguridad. El resultado no es únicamente un dispositivo que mide variables eléctricas, sino un sistema completo capaz de observar, interpretar y actuar sobre una instalación eléctrica.

Uno de los principales aportes del proyecto es que el módulo físico mantiene capacidad de protección aun cuando la conectividad falla. El ESP32-C3 puede evaluar localmente una condición crítica y accionar el relevador sin depender de Internet o del servidor. Paralelamente, el backend implementa un segundo nivel de reglas que permite registrar eventos, bloquear dispositivos y conservar trazabilidad sobre lo ocurrido. Esta arquitectura de dos niveles mejora la confiabilidad del sistema porque una falla de comunicación no elimina la función básica de protección eléctrica.

Otro aspecto relevante es que la plataforma no depende de una comunicación directa entre el teléfono, la computadora y el dispositivo físico. Todos los clientes convergen en el backend central, que funciona como punto de coordinación y fuente de verdad del sistema. Esta decisión reduce el acoplamiento entre componentes y permite que el firmware, el dashboard web o la aplicación móvil evolucionen de manera independiente mientras respeten el mismo contrato de comunicación.

La separación de las APIs en superficies específicas para IoT, usuarios y administración aporta además una estructura clara de seguridad y responsabilidades. El dispositivo físico únicamente tiene acceso a las operaciones necesarias para reportar telemetría y recibir comandos; el usuario final puede acceder solamente a sus sitios y dispositivos; y el administrador dispone de herramientas globales de gestión. Esta separación reduce el riesgo de exposición accidental de funciones privilegiadas y facilita el mantenimiento de la plataforma.

El modelo de datos también constituye una mejora importante respecto a soluciones IoT básicas. VoltGuard no relaciona simplemente un usuario con un dispositivo, sino que incorpora sitios, membresías, perfiles de seguridad, credenciales, telemetría, comandos, eventos, notificaciones, tarifas y predicciones. Esto permite representar escenarios reales como una casa, laboratorio, taller o negocio con múltiples usuarios y múltiples dispositivos dentro de una misma instalación.

Una decisión especialmente relevante fue separar el estado deseado del estado real de cada dispositivo. Cuando un usuario solicita apagar una carga, el sistema no asume que la acción se ejecutó únicamente porque el servidor generó la orden. El dispositivo confirma posteriormente la ejecución mediante un ACK, permitiendo

distinguir entre lo solicitado y lo realmente realizado. Esto mejora la confiabilidad del control remoto y permite detectar fallas de comunicación o ejecución.

La combinación de WebSocket y HTTP también aporta robustez. WebSocket permite entregar órdenes con baja latencia, mientras que el mecanismo de consulta periódica por HTTP funciona como respaldo. Si el canal en tiempo real falla, la orden permanece disponible y puede recuperarse posteriormente. Esta decisión evita depender de un único mecanismo de comunicación y mejora la tolerancia a fallos. Durante pruebas con hardware real se registraron latencias de comando de aproximadamente 0.27 a 0.53 segundos después de aislar el cliente WebSocket en una tarea independiente de FreeRTOS.

VoltGuard también tiene impacto desde el punto de vista del usuario. La información obtenida por el PZEM-004T deja de ser una serie de valores aislados y se transforma en información útil: consumo actual, histórico, costo, eventos, predicción y anomalías. Tanto la aplicación móvil como el dashboard web permiten observar y controlar los dispositivos sin necesidad de comprender el funcionamiento interno del hardware.

La incorporación de funciones administrativas amplía el alcance del proyecto más allá de una instalación individual. La plataforma puede gestionar múltiples usuarios, sitios y dispositivos, así como visualizar métricas globales, dispositivos conectados o desconectados, anomalías y eventos críticos. Esto permite considerar VoltGuard no solamente como un prototipo doméstico, sino como una base para aplicaciones en laboratorios, pequeños comercios, talleres, instalaciones educativas o entornos con múltiples cargas supervisadas.

El análisis de datos añade otra capa de valor. El sistema no se limita a almacenar telemetría, sino que utiliza el histórico para realizar proyecciones de consumo y detectar comportamientos anómalos. Se eligieron modelos relativamente simples y explicables, como tendencia lineal, z-score e Isolation Forest, evitando introducir complejidad artificial cuando todavía no existe suficiente información que justifique modelos más avanzados.

También resulta relevante que el sistema haya sido validado con hardware real y no únicamente mediante simulaciones. El reporte documenta un dispositivo físico conectado, con lecturas reales de voltaje, corriente y consumo obtenidas mediante el PZEM-004T y mostradas posteriormente en las interfaces de usuario. Esto demuestra que la integración entre hardware, firmware, red, backend, base de datos y aplicaciones funciona de extremo a extremo.

Desde el punto de vista de ingeniería de software, el proyecto también incorpora prácticas propias de un entorno profesional. Se utilizaron migraciones de base de datos, contenedores, integración continua, análisis estático, pruebas automatizadas y control de cobertura. La versión documentada cuenta con 110 pruebas automatizadas y aproximadamente 94 % de cobertura, lo que reduce el riesgo de introducir regresiones durante la evolución del sistema.

Impacto del proyecto

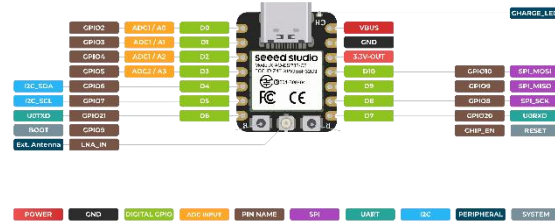
El impacto principal de VoltGuard consiste en transformar el consumo eléctrico de una variable normalmente invisible para el usuario en información observable, histórica y accionable. En lugar de conocer el consumo únicamente cuando llega un recibo, el usuario puede identificar en tiempo real cuánto consume una carga, consultar su comportamiento histórico y detectar incrementos anormales.

A esto se agrega un segundo impacto más relevante: VoltGuard no solamente informa, sino que puede actuar. Ante una condición crítica, el sistema puede interrumpir físicamente la alimentación de una carga. Esta combinación entre monitoreo y actuación convierte al proyecto en una plataforma de protección y gestión energética, no únicamente en un sistema de telemetría.

El proyecto también demuestra que es posible construir una solución distribuida completa a partir de hardware relativamente accesible. Un ESP32-C3 y un módulo de medición eléctrica se integran con tecnologías de backend, bases de datos, aplicaciones web y móviles, comunicaciones en tiempo real y técnicas de análisis de datos, formando una plataforma que puede evolucionar hacia escenarios de mayor escala.

Finalmente, VoltGuard tiene relevancia porque aborda simultáneamente tres necesidades: seguridad eléctrica, conocimiento del consumo y control remoto. La integración de estas funciones dentro de una misma arquitectura permite reducir riesgos, comprender mejor el comportamiento energético de los equipos y facilitar la administración de múltiples cargas desde una interfaz centralizada.

Microcontrolador: Seeed Studio XIAO ESP32-C3



La elección se mantuvo durante todo el desarrollo debido a que integra:

- ESP32-C3;
- Wi-Fi;
- lógica de 3.3 V;
- tamaño reducido;
- UART;
- GPIO suficientes para el prototipo;
- capacidad de ejecutar la lógica de medición, protección y comunicación.

Los pines finalmente utilizados fueron:

Función	Pin XIAO	GPIO
Control del relé	D2	GPIO4
UART TX hacia PZEM	D6	GPIO21
UART RX desde PZEM	D7	GPIO20
Alimentación	5V	—
Tierra	GND	—

Componentes de Medición:

Inicialmente evaluamos construir la etapa de medición a partir de diferentes componentes especializados:

- MCP39F511A;
- ZMPT101L para tensión;
- SCT-013 u otro transformador de corriente;
- circuitería adicional de acondicionamiento.

El MCP39F511A era particularmente atractivo debido a su orientación a medición energética, pero encontramos dificultades importantes de disponibilidad dentro del tiempo de adquisición disponible.

Mantener sensores independientes también implicaba:

- mayor número de componentes;
- mayor cantidad de etapas analógicas;
- calibración individual;
- mayor complejidad de PCB;
- mayor riesgo durante el desarrollo.

Decidimos sustituir esa arquitectura por una más ligera de mayor facilidad para adquirir los componentes electrónicos.

PZEM-004T-100A V4.0

Esta decisión redujo considerablemente la complejidad, ya que el mismo módulo proporciona:

- tensión;
- corriente;
- potencia activa;
- factor de potencia;
- frecuencia;
- energía acumulada.



La versión de 100 A utiliza un transformador de corriente externo.

Por esta razón eliminamos del diseño:

- ZMPT101L;
- SCT-013 independiente;
- MCP39F511A.

El PZEM pasó a representar el subsistema completo de medición energética.

El CT del PZEM no se conecta eléctricamente en serie con la carga. El conductor de fase simplemente atraviesa físicamente el núcleo del transformador:

Únicamente debe atravesar el CT el conductor de fase. No deben atravesarlo simultáneamente fase y neutro porque sus campos magnéticos se cancelarían aproximadamente:

$$I_{\text{medida}} \approx I_L - I_N$$

En una carga normal:

$$I_L \approx I_N$$

por lo que:

$$I_{\text{medida}} \approx 0$$

aunque realmente exista consumo.

Los dos cables de señal del CT se conectan a:

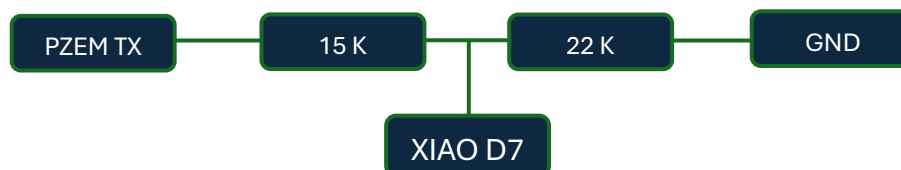


Antes de conectar directamente el UART del PZEM a la XIAO realizamos mediciones eléctricas. Encontramos:

- PZEM VCC ≈ 5 V
- PZEM TX en reposo ≈ 5 V

Esto fue relevante porque el ESP32-C3 trabaja con lógica de 3.3 V y no debemos aplicar directamente una señal de aproximadamente 5 V a sus GPIO. Por esta razón descartamos la conexión directa.

Para reducir la salida de aproximadamente 5 V del PZEM utilizamos un divisor resistivo:



La ecuación utilizada para calcular los valores de resistencia es:

$$V_{out} = V_{in} \frac{R_2}{R_1 + R_2}$$

Sustituyendo:

$$\begin{aligned} V_{out} &= 5 \frac{22\,000}{15\,000 + 22\,000} \\ V_{out} &= 5 \frac{22}{37} \\ V_{out} &\approx 2.97\,V \end{aligned}$$

Experimentalmente obtuvimos:

$$V_{RX} \approx 2.97\,V$$

por lo que validamos el divisor.

La corriente aproximada que circula permanentemente por el divisor cuando TX se encuentra a 5 V es:

$$\begin{aligned} I &= \frac{V}{R} \\ I &= \frac{5}{15\,000 + 22\,000} \\ I &\approx 135\,\mu A \end{aligned}$$

Se trata de una corriente suficientemente pequeña y la comunicación a 9600 baudios funcionó correctamente durante las pruebas.

En la dirección contraria teníamos:

$$XIAO\,TX \approx 3.3\,V$$

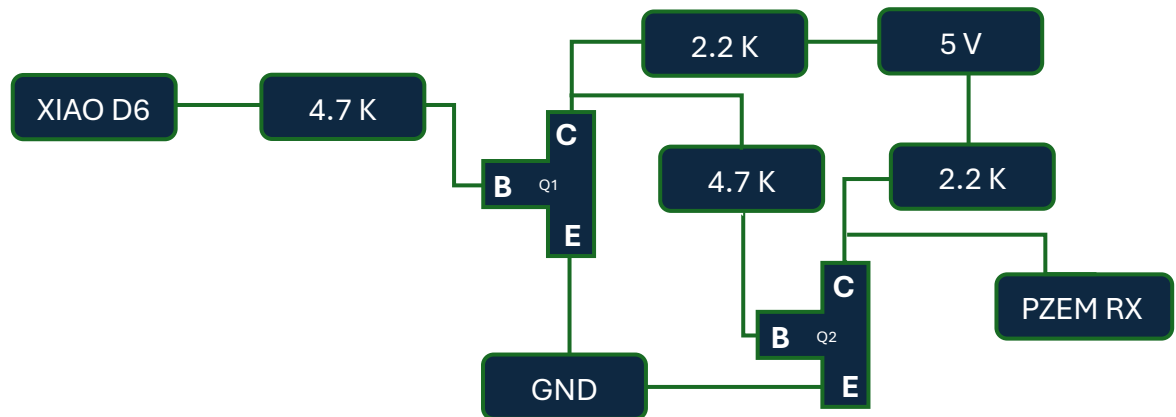
PZEM RX → interfaz de aproximadamente 5 V

De los 3.3 V que nos da la XIAO, se necesitan 5V para el PZEM, por lo que no es suficiente lo que la da XIAO, había que subir el voltaje. Inicialmente consideramos utilizar un integrado de adaptación lógica como el 74HCT125.

Sin embargo, debido a los componentes disponibles y a que solo necesitábamos una línea UART, construimos una adaptación discreta con dos transistores 2N2222A.

La utilización de dos etapas era necesaria porque una sola etapa con transistor NPN invertiría la lógica UART.

La arquitectura adoptada fue:



Las dos inversiones restauran la polaridad original:

D6 LOW → Q1 salida HIGH → Q2 salida LOW

D6 HIGH → Q1 salida LOW → Q2 salida HIGH

Experimentalmente obtuvimos aproximadamente:

D6 LOW → PZEM RX ≈ 0–0.2 V

D6 HIGH → PZEM RX ≈ 5 V

La interfaz quedó validada.

La corriente máxima aproximada del pull-up de 2.2 kΩ es:

$$I = \frac{5}{2200}$$
$$I \approx 2.27 \text{ mA}$$

La corriente de base aproximada de Q1 con resistencia de 4.7 kΩ es:

$$I_B = \frac{3.3 - 0.7}{4700}$$

$$I_B \approx 0.55 \text{ mA}$$

La relación proporciona margen suficiente para conmutar los aproximadamente 2.27 mA requeridos por la resistencia pull-up.

Fuente de alimentación Mean Well IRM-20-5

Para poder alimentar el circuito correspondiente a DC desde el mismo módulo, tendríamos que transformar la corriente AC a DC, para esto decidimos usar una mini fuente conmutada Mean Well IRM-20-5 la cual recibe 127 V AC y los convierte a 5V DC para alimentar la ESP32-C3.



La fuente instalada dispone de:

- Salida nominal: 5 V
- Corriente nominal: 4 A
- Potencia nominal: 20 W

Esto proporciona margen considerable respecto al consumo de:

- ESP32-C3;
- interfaz PZEM;
- módulo de relé;
- circuitería discreta.

No pretendemos consumir 4 A; disponer de una fuente con mayor capacidad simplemente evita trabajar cerca de su límite. Durante las pruebas consideramos alimentar directamente el pin de 5 V de la XIAO desde la IRM. La documentación de Seeed indica que al utilizar una alimentación externa sobre dicho pin debe existir un diodo entre la fuente externa y la entrada de 5 V. Solo disponíamos inicialmente de 1N4007 y realizamos la prueba y encontramos aproximadamente:

- Antes del 1N4007 $\approx 5.0 \text{ V}$
- Después del 1N4007 $\approx 4.2 \text{ V}$

La caída fue de aproximadamente:

$$V_F \approx 0.8 \text{ V}$$

Por ello, para el prototipo retiramos temporalmente el 1N4007 y alimentamos directamente la entrada de 5 V. Esto se aceptó únicamente con la condición de que la IRM alimentando XIAO NUNCA se conectara simultáneamente por USB a la computadora.

- diodo Schottky de baja caída, por ejemplo 1N5819 o SS14, después de verificar corriente y temperatura;

- una solución apropiada de power-path/ORing que impida cualquier retorno de corriente hacia USB.

T90-5VDC-TL-C

- fusible;
- portafusible;
- pistas del módulo;
- terminales;
- conectores;
- calibre del conductor;
- temperatura;
- PCB;

Nuestro prototipo actualmente queda limitado a 10 A, independientemente del valor que acepte el relé. Inicialmente conectamos el GPIO de la XIAO directamente a la entrada IN del módulo T90. Observamos que $\text{GPIO} \approx 3.265 \text{ V}$, y encendía el LED del módulo, pero no producía de forma fiable la activación mecánica del relevador. Al conectar manualmente IN a 5 V, el relé sí conmutaba.

Concluimos que no debíamos depender de que una salida de 3.3 V alcanzara correctamente el nivel requerido por el circuito del módulo.

Por ello configuramos el módulo para:

Trigger = LOW

y añadimos un transistor 2N2222A como colector abierto se agregó una resistencia de pull-up externo de 4.7 k Ω se añadió después de observar que IN no regresaba correctamente a 5 V al apagar el transistor.

La corriente del pull-up es aproximadamente:

$$I = \frac{5}{4700}$$
$$I \approx 1.06 \text{ mA}$$

Cuando el GPIO se encuentra HIGH $D2 \approx 3.3 \text{ V}$, Q_RELE conduce y lleva $IN \approx 0 \text{ V}$ por lo tanto: Relé ON

Cuando D2 se encuentra LOW, Q_RELE apagado y el pull-up lleva $IN \approx 5 \text{ V}$ por lo tanto: Relé OFF

```
pinMode(RELAY_CTRL, OUTPUT);  
digitalWrite(RELAY_CTRL, LOW);
```

Esta arquitectura quedó validada experimentalmente.

Configuramos inicialmente D2 en estado bajo para que el relé permanezca desactivado durante el arranque.

El principio utilizado es:

Nuestro objetivo es evitar que un reinicio del microcontrolador produzca una activación accidental de la carga.

La política prevista será fail-safe, es decir, ante una condición indeterminada debemos definir explícitamente si la carga debe desconectarse.

Decidimos conmutar únicamente la fase. Utilizamos el contacto NO, normalmente abierto, para que la carga permanezca desconectada cuando el relé no está energizado. No utilizamos NC y el neutro va directamente:



Y cuando exista conductor de protección:



La tierra física nunca debe ser conmutada.

Conectamos la medición de tensión del PZEM antes del relé:



mientras la corriente de la carga se mide después del relé mediante el CT. Esto permite distinguir:

Red presente + carga apagada

de:

Red ausente

Cuando abrimos el relé esperamos:

- Voltaje $\approx 125\text{ V}$
- Corriente $\approx 0\text{ A}$
- Potencia $\approx 0\text{ W}$

Este comportamiento fue verificado. Conectamos inicialmente únicamente:



sin una carga atravesando el CT.

La primera lectura estabilizada fue:

- Voltaje: 125.2 V
- Corriente: 0.000 A
- Potencia: 0.0 W
- Frecuencia: 59.9 Hz
- Factor de potencia: 0.00
- Energía acumulada: 2 Wh

Estos resultados fueron coherentes. Los 2 Wh no representaron una falla, ya que el PZEM mantiene un contador de energía acumulada. No reiniciamos dicho contador durante estas pruebas. Posteriormente conectamos una carga pequeña y pasamos únicamente su conductor de fase por el CT.

Obtuvimos:

- Corriente: 0.096 A
- Potencia: 8.3 W
- Frecuencia: 60.0 Hz
- Factor de potencia: 0.69
- Energía acumulada: 2 Wh

Tomando aproximadamente:

$$V = 125.2 \text{ V}$$

la potencia aparente es:

$$\begin{aligned} S &= VI \\ S &= 125.2(0.096) \\ S &\approx 12.02 \text{ VA} \end{aligned}$$

La potencia activa esperada es:

$$P = S \cdot PF$$

$$P = 12.02(0.69)$$

$$P \approx 8.29 \text{ W}$$

El PZEM reportó:

$$P = 8.3 \text{ W}$$

por lo que la lectura quedó matemáticamente validada.

La diferencia entre 12 VA de potencia aparente y aproximadamente 8.3 W de potencia activa se explica por:

$$PF \approx 0.69$$

resultado compatible con una carga que no sea puramente resistiva. Posteriormente integramos ambos subsistemas. Programamos el relevador para cambiar de estado periódicamente.

Cuando:

RELE ON: observamos una carga encendida con Corriente > 0 y Potencia > 0

RELE OFF: observamos una carga apagada con Corriente ≈ 0 y Potencia ≈ 0

Mientras, en cualquier caso, Voltaje $\approx 125 \text{ V}$ y Frecuencia $\approx 60 \text{ Hz}$

Fusibles

Inicialmente buscábamos un fusible T2AH250V, es decir, un fusible retardado y de corte rápido. No fue posible conseguirlo dentro del tiempo disponible. Sólo pudimos conseguir de:

- F1AL250V
- F10AH250V
- F15AL250V

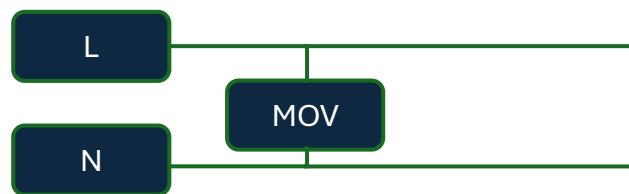
Para las pruebas utilizamos F1AL250V en la rama de la IRM/PZEM. El fusible sobrevivió los arranques realizados. Sin embargo, F1AL250V debe considerarse una solución de prototipo. Su característica rápida puede provocar aperturas no deseadas debido a la corriente de irrupción de la fuente. Para la versión final debemos seleccionar un fusible temporizado adecuado a:

- corriente real de electrónica;
- corriente de inrush;
- capacidad de ruptura;
- temperatura;
- normativa;
- portafusible utilizado.

Varistor

Consideramos añadir un MOV entre fase y neutro para limitar transitorios de sobretensión.

El MOV debe conectarse:



No se instala en serie con la carga.

Además, debe colocarse después de una protección fusible, ya que un MOV degradado puede fallar en cortocircuito. Disponemos físicamente del varistor 14D201K. Sus parámetros relevantes son aproximadamente:

- Máxima tensión continua AC: 130 Vrms
- Tensión nominal de varistor: 200 V
- Clamp aproximado: 340 V

La red nominal utilizada es:

$$127V_{RMS}$$

El margen respecto a 130 V es únicamente:

$$130 - 127 = 3V$$

Este margen resulta demasiado pequeño.

Además, el pico sinusoidal correspondiente a 127 Vrms es:

$$V_{pico} = V_{RMS}\sqrt{2}$$

$$V_{pico} = 127(1.414)$$

$$V_{pico} \approx 179.6V$$

Aunque un MOV no se selecciona únicamente mediante esta comparación directa, la tensión RMS máxima permitida muestra claramente que el 14D201K trabaja demasiado cerca de la tensión nominal de nuestra red. Por ello no seleccionamos el 14D201K para la versión final.

También disponemos del varistor 14D391K. Sus valores aproximados son:

- Máxima tensión AC continua: 250 V
- Tensión nominal de varistor: 390 V
- Clamp aproximado: 650 V

Este componente puede permanecer conectado a una red de 127 V sin encontrarse cerca de su tensión máxima continua. Sin embargo, para nuestro objetivo de protección actuaría a tensiones excesivamente altas. Por ello tampoco seleccionamos el 14D391K como MOV principal para la red de 127 V. Es una elección más acorde con líneas de tensión superiores.

Identificamos como candidato 14D241K con valores aproximadamente:

- Máxima tensión AC continua: 150 V
- Tensión nominal de varistor: 240 V
- Clamp aproximado: 395 V
- Corriente de pulso especificada: hasta 4.5 kA bajo las condiciones de ensayo del fabricante
- Proporciona mayor margen respecto a 127 V que el 14D201K sin elevar tanto el nivel de protección como el 14D391K.

Sin embargo, no lo encontramos disponible por lo que para las pruebas no usamos varistor. Para un diseño de producto final también debemos considerar una protección térmica del MOV o un componente tipo TMOV, ya que los MOV pueden degradarse térmicamente después de múltiples transitorios.

Software

Desarrollamos el firmware en Arduino para ESP32-C3. Para comunicación con el PZEM utilizamos la librería: *MycilaPZEM*

La UART se configuró mediante Serial1 y los pines utilizados fueron:

PZEM_TX = D6

PZEM_RX = D7

Desde la perspectiva de la XIAO:

D6 = transmisión

D7 = recepción

En el prototipo utilizamos la dirección 0xF8 y la comunicación quedó funcional con la librería utilizada.

Si posteriormente cambiamos de librería o implementamos Modbus directamente, deberemos revisar nuevamente la semántica de direccionamiento y no asumir que 0xF8 se comportará de la misma manera en cualquier implementación.

Validamos primero la IRM sin conectar ningún circuito a su salida. La entrada se conectó a 127 VCA y verificamos:

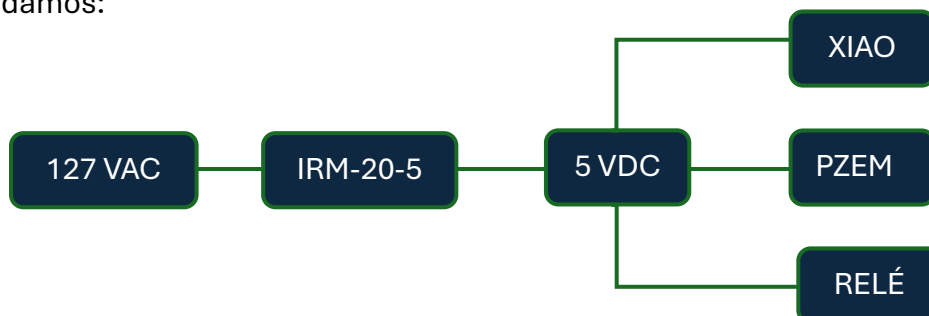
$$V_{out} \approx 5V$$

Posteriormente conectamos:

1. XIAO;
2. PZEM;
3. relé.

Las tensiones permanecieron estables.

Finalmente retiramos la alimentación USB y verificamos que todo el sistema pudiera iniciar utilizando únicamente la IRM. La prueba fue satisfactoria. Por lo tanto, validamos:



La IRM-20-5 dispone de una eficiencia típica de aproximadamente 79 % en la condición especificada por el fabricante para el modelo de 5 V. Si utilizáramos los 20 W completos:

$$P_{in} = \frac{P_{out}}{\eta}$$
$$P_{in} = \frac{20}{0.79}$$
$$P_{in} \approx 25.3W$$

A 127 V:

$$I_{in} \approx \frac{25.3}{127}$$
$$I_{in} \approx 0.20A$$

Por lo tanto, el consumo estacionario de la fuente es muy inferior a 1 A. Sin embargo, esto no significa que un fusible rápido de 1 A sea automáticamente la selección final adecuada, porque la IRM presenta corriente de arranque considerable. El fabricante especifica un inrush típico del orden de 20 A durante arranque en frío a 115 V. Por ello distinguimos corriente continua normal de corriente transitoria de arranque.

4. Metodología

4.1. Diseño de Módulo

Adoptamos un desarrollo modular y progresivo. Se desarrolló todo el módulo por subsistemas, cada subsistema se sometió a pruebas individuales antes de incorporarse al circuito completo.

La secuencia utilizada fue:

1. validar alimentación y GPIO de la XIAO;
2. validar el módulo de relé únicamente en baja tensión;
3. caracterizar eléctricamente la interfaz UART del PZEM;
4. desarrollar adaptación de niveles lógicos;
5. validar la fuente IRM-20-5;
6. alimentar conjuntamente los subsistemas de baja tensión;
7. conectar el PZEM a la red sin carga;

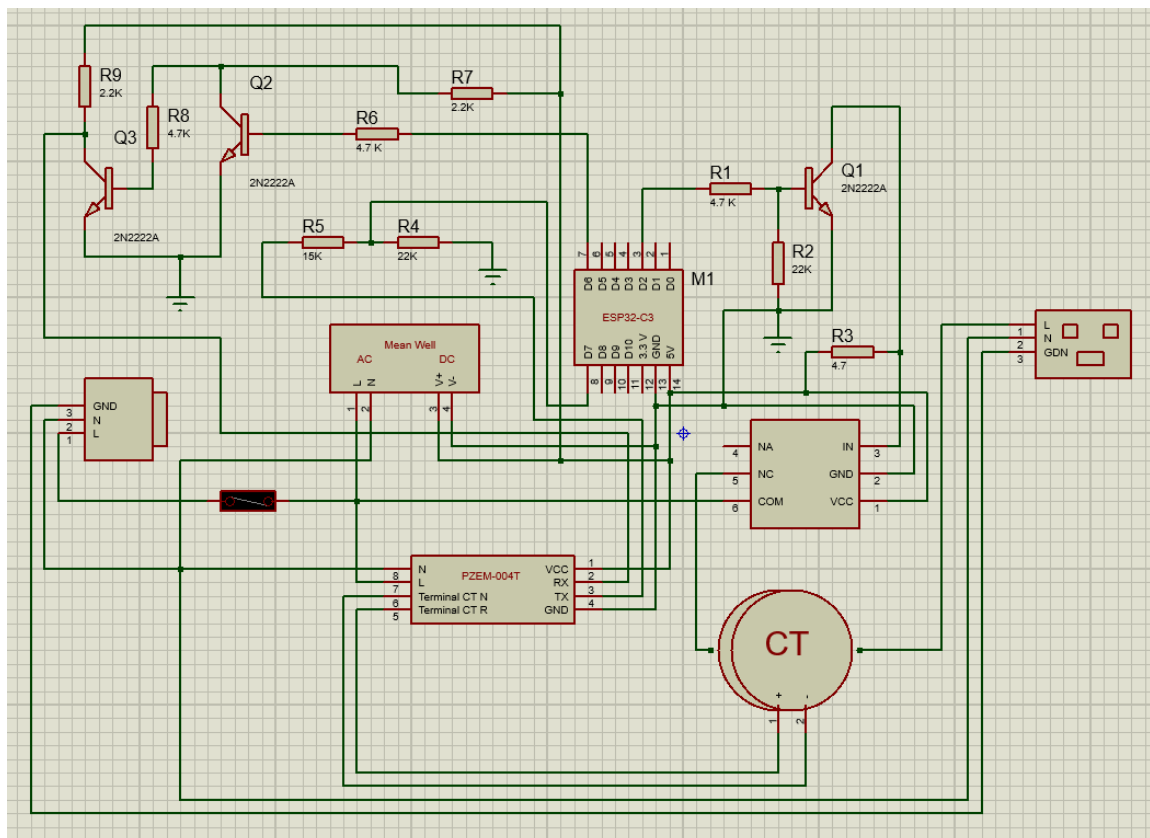
8. medir una carga pequeña mediante el CT;
9. insertar el relé en la fase;
10. combinar medición y conmutación;
11. eliminar la alimentación USB y utilizar la IRM;
12. revisar la protección mediante fusibles;
13. incorporar el diseño de protección contra transitorios mediante MOV;
14. preparar posteriormente la capa Wi-Fi, servidor y aplicación.

Este método redujo el riesgo de dañar componentes y facilitó localizar las fallas en cada etapa.

El prototipo validado se compone de:

- 1 × Seeed Studio XIAO ESP32-C3
- 1 × PZEM-004T-100A V4.0
- 1 × CT correspondiente al PZEM
- 1 × IRM-20-5
- 1 × módulo T90-5VDC-TL-C
- 3 × 2N2222A
- F1AL250V

El diseño del diagrama esquemático se realizó en Proteus y queda de la siguiente forma:



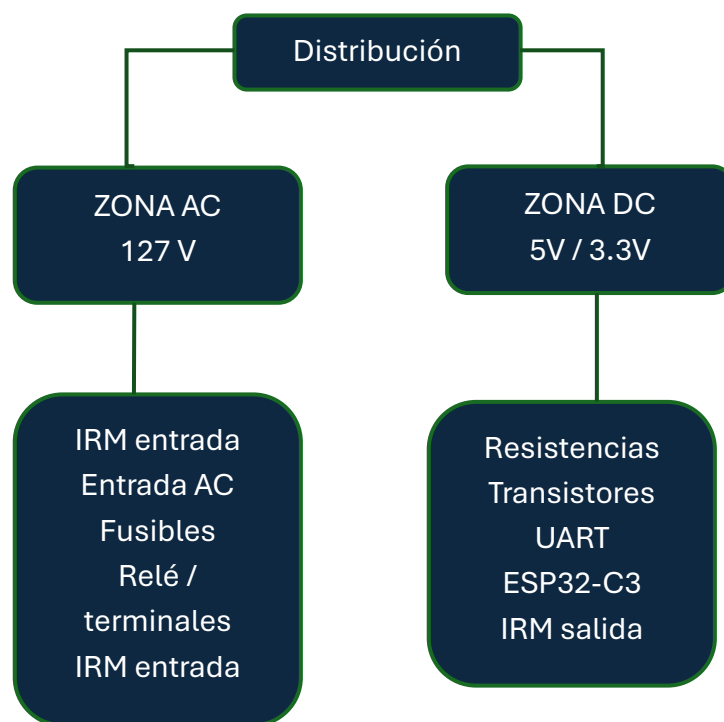
Como parte de la evolución del prototipo hacia una implementación física más ordenada y segura, diseñamos una placa PCB específica para integrar la electrónica principal del sistema.

La PCB tiene dimensiones aproximadas de:

8 cm × 7 cm

El diseño de la placa se realizó considerando desde el inicio la separación entre la sección conectada a tensión de red y la sección de baja tensión.

La distribución general se planteó como:



Dividimos físicamente la PCB en dos regiones:

- zona de alta tensión AC;
- zona de baja tensión DC.

La zona AC contiene o interconecta los elementos asociados a:

- entrada de 127 VCA;
- fase;
- neutro;
- fusibles;

- entrada de la IRM-20-5;
- contactos de potencia del relé;
- conexiones hacia la carga;
- PZEM en su sección de medición de red.

La zona DC contiene principalmente:

- alimentación de 5 V;
- señales de 3.3 V;
- resistencias;
- transistores 2N2222A;
- señales UART;
- control del relé;
- señales asociadas al ESP32-C3.

Esta separación tiene como objetivo disminuir el riesgo de interacción accidental entre la red eléctrica y la electrónica de control, además de facilitar el mantenimiento y la inspección del sistema.

Para la versión final debemos continuar verificando que las distancias entre pistas, pads, terminales y elementos de AC y DC cumplan los valores de creepage y clearance correspondientes a la tensión, grado de contaminación, material de PCB y categoría de sobretensión de la aplicación.

Decidimos no soldar todos los módulos directamente sobre la PCB.

La placa se utiliza principalmente como:

- soporte estructural;
- distribución eléctrica;
- interconexión;
- acondicionamiento de señales;
- separación organizada entre AC y DC.

Los componentes básicos de acondicionamiento sí se soldarán directamente sobre la tarjeta. Entre ellos: Resistencias, Transistores 2N2222A, Elementos pasivos auxiliares, Conectores y terminales. Esto permite que las etapas discretas del circuito permanezcan fijas y con conexiones cortas.

Los principales módulos del sistema se conectarán mediante terminales y no quedarán soldados permanentemente a la PCB. Entre ellos se encuentran: XIAO

ESP32-C3, PZEM-004T-100A V4.0, módulo de relé T90, transformador de corriente CT, conexiones de entrada y salida de red y carga externa.

La interconexión mediante terminales permite:

- desmontar componentes sin desoldar;
- sustituir módulos dañados;
- realizar pruebas individuales;
- facilitar mantenimiento;
- modificar futuras versiones;
- reutilizar la PCB durante el desarrollo.

De esta manera evitamos convertir un fallo de un módulo comercial en la necesidad de reemplazar toda la tarjeta. La decisión de no soldar permanentemente todos los módulos aporta ventajas importantes durante el desarrollo.

Mantenimiento: Si por ejemplo falla el PZEM, podemos reemplazarlo sin intervenir la electrónica discreta.

Actualización: Si posteriormente sustituimos la XIAO ESP32-C3 por otro microcontrolador compatible, podemos rediseñar únicamente la interfaz correspondiente.

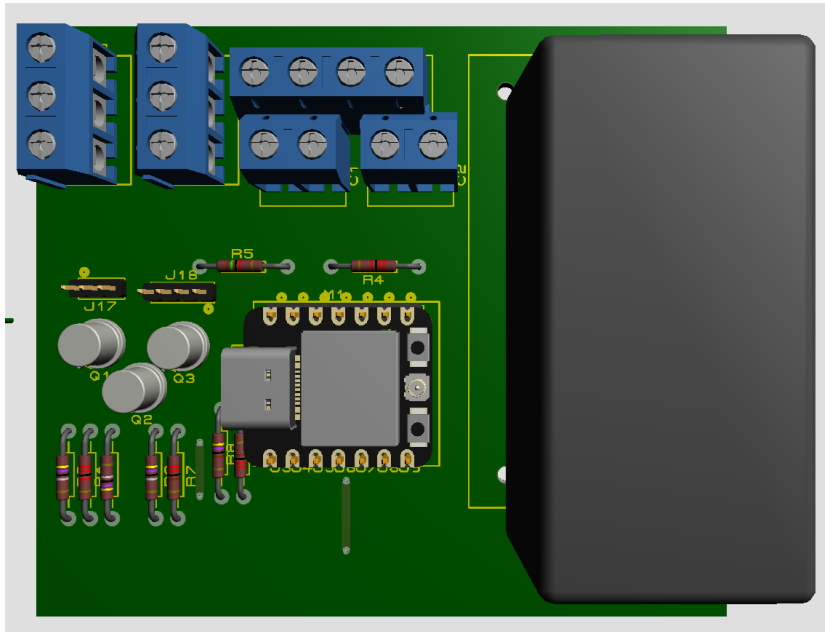
Diagnóstico: Podemos desconectar individualmente:

- PZEM
- Relé
- ESP32
- CT

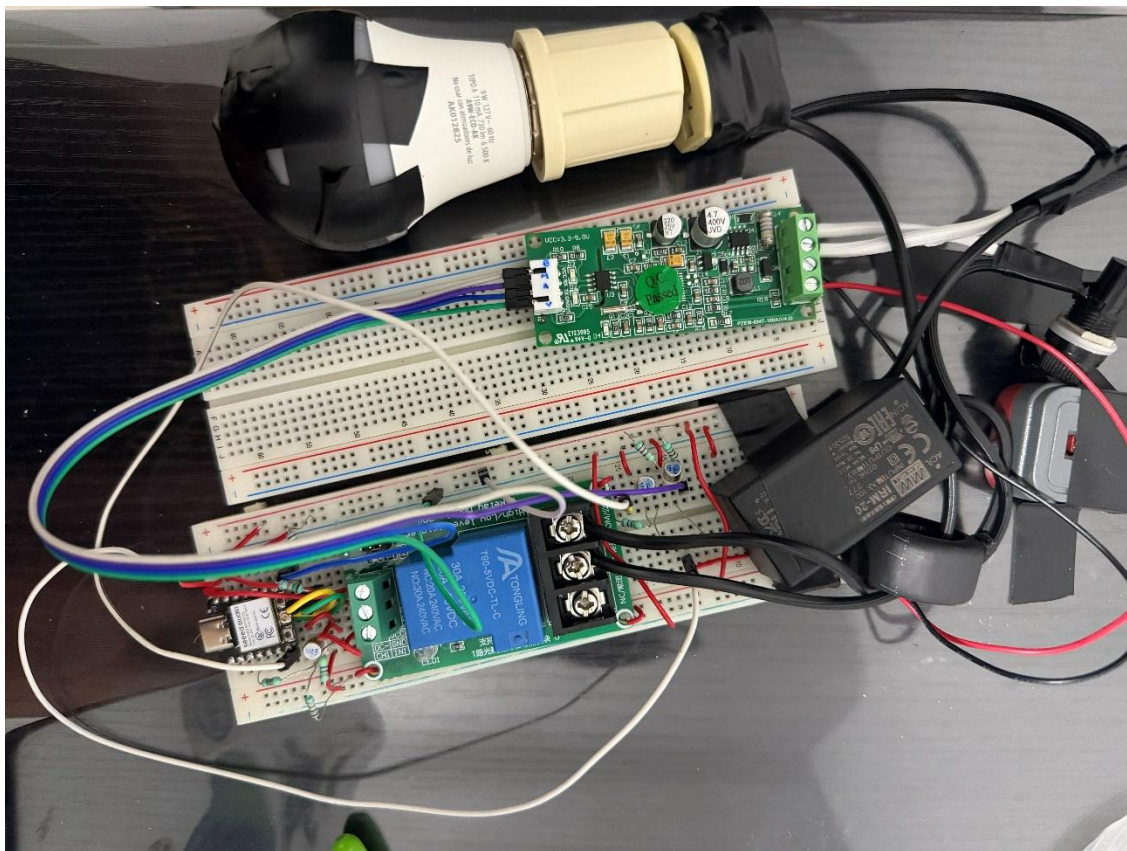
para realizar pruebas aisladas.

Reutilización: La tarjeta principal puede mantenerse incluso si se modifica alguno de los módulos externos.

La vista pronosticada de cómo se vería la PCB, es la siguiente:

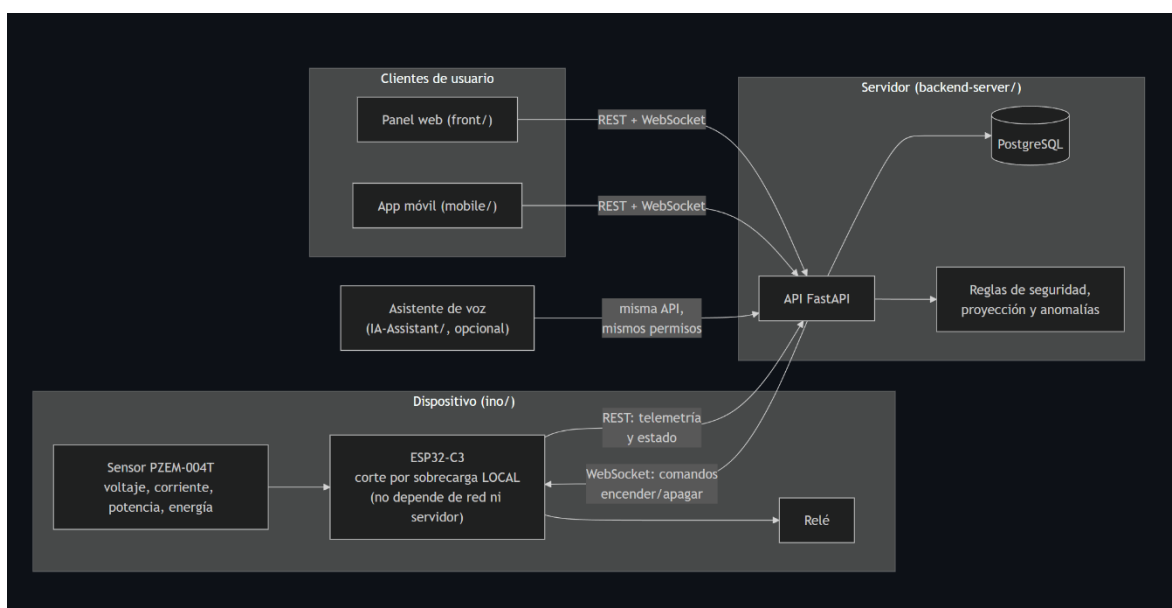


El prototipo con el que se hicieron pruebas fue con la circuitería montada en Protoboard como se muestra a continuación.



4.2. Arquitectura general de la plataforma

La arquitectura se puede resumir en una regla simple, aplicada sin excepciones: ningún cliente le habla directamente a otro cliente. El firmware nunca le habla al navegador ni al celular; el navegador nunca le habla al firmware. Todo converge en el backend, que es la única pieza que conoce el estado completo y verdadero del sistema.



Esta decisión de diseño tiene una consecuencia directa e importante: cada pieza puede evolucionar, romperse o reemplazarse sin arrastrar a las demás. Si el dashboard web cambia por completo su interfaz, el firmware no se entera ni necesita ningún cambio. Si se agrega una segunda app móvil, no hace falta tocar una sola línea del backend existente — solo consumir el mismo contrato de API que ya usan front/ y mobile/ .

El repositorio es un monorepo: cuatro proyectos independientes, cada uno con sus propias dependencias y su propio ciclo de build, conviviendo en el mismo historial de control de versiones (front/ , mobile/ , ino/), más una capa de infraestructura de despliegue (backend-server/ , infra/ , docker-compose.yml) y un módulo opcional y desacoplable de asistente de voz (IA-Assistant/).

·	
└─ backend-server/	Backend FastAPI + PostgreSQL
└─ app/	
└─ api/{iot,app,admin}/	Las tres superficies de API
└─ core/	Config, seguridad, WS manager, dependencias
└─ models/entities.py	Modelo de datos único
└─ schemas/	Contratos Pydantic, uno por superficie
└─ services/	Lógica de negocio pura
└─ tests/	Batería pytest (110 pruebas)
└─ alembic/	Migraciones de base de datos
└─ ino/	Firmware ESP32-C3
└─ front/	Dashboard web – React + Vite + TS
└─ mobile/	App Android – React + Capacitor
└─ IA-Assistant/	Asistente de voz, opcional y desacoplado
└─ infra/	Proxy inverso y TLS (Caddy)
└─ docker-compose.yml	Despliegue de plataforma completa
└─ docs/	Documentación de referencia y este reporte

El firmware: capa de red sobre hardware validado

El trabajo sobre el firmware se organizó como una capa agregada, nunca como un reemplazo, de la lógica de medición y control ya validada en banco de pruebas. El archivo principal (code.ino) conserva intacta la lectura del sensor PZEM-004T (vía la librería MycilaPZEM , asíncrona) y el control directo del relé. Sobre eso se añadieron módulos nuevos, cada uno con una responsabilidad única:

Módulo	Por qué existe
local_safety	Aplica el corte de corriente evaluando el umbral guardado en memoria flash (NVS), sin hacer ninguna llamada de red . Es la garantía de seguridad que no puede fallar por un problema de conectividad.
provisioning	Resuelve el problema de "cómo le doy WiFi y credenciales a un dispositivo que todavía no tiene red": levanta su propio punto de acceso temporal con un portal de configuración la primera vez que se enciende.
device_storage	Centraliza toda la persistencia en NVS (credenciales WiFi, URL del backend, identidad y secreto del dispositivo, umbral de corte).
iot_client	Cliente HTTP: reporta telemetría, hace <i>poll</i> de comandos pendientes cada 3 segundos, manda heartbeat, confirma comandos aplicados. Es la vía "siempre disponible", de respaldo.
iot_ws_client	Cliente WebSocket, ejecutado en su propia tarea de FreeRTOS. Recibe comandos empujados por el backend casi al instante.

La decisión de diseño más importante de todo el firmware es que la única garantía dura de seguridad no depende de la red. Si el WiFi se cae, si el backend está

caído, o si hay latencia de varios segundos, el dispositivo sigue midiendo corriente y sigue siendo capaz de cortar la carga por sí solo. Todo lo demás (recibir comandos remotos, sincronizar un umbral nuevo) es un plus, es decir, mejora la experiencia cuando hay red, pero su ausencia nunca compromete la seguridad eléctrica de la instalación.

Cliente WebSocket corre en una tarea separada. Esto no fue una decisión anticipada, sino una corrección aplicada tras medir un problema real: al integrar el cliente WebSocket en el mismo bucle que ya usaba el cliente HTTP, un intento de reconexión fallido de la librería podía bloquear el hilo principal del microcontrolador durante varios segundos, degradando también la lectura del sensor. La solución fue mover ese cliente a su propia tarea de FreeRTOS, de forma que un problema de reconexión del canal en tiempo real nunca afecte al resto del sistema. Latencia de comando medida en hardware real tras el arreglo: 0.27–0.53 segundos.

El backend: diseño del núcleo del sistema

El backend es la pieza central y la que concentra la mayor parte de la complejidad del proyecto. Está construido en Python 3.12 con FastAPI, SQLAlchemy como capa de acceso a datos, Alembic para versionar el esquema de base de datos, y PostgreSQL como motor de base de datos. Se eligió FastAPI por tres razones concretas que se aprovechan activamente: genera automáticamente un documento OpenAPI a partir del propio código (que después se usa para generar los tipos de TypeScript de front/ y mobile/ sin escribirlos a mano), valida entradas y salidas de forma declarativa, y tiene soporte nativo para WebSocket sobre el mismo framework que las rutas REST.

El modelo de datos (app/models/entities.py) resuelve el problema de varias personas, varios lugares, varios dispositivos, y que cada uno tenga sus propias reglas de seguridad. La clasificación queda de la siguiente manera:

Identidad y acceso

Entidad	Columnas relevantes	Justificación de diseño
User	email (único), hashed_password , role (user / admin), is_active	El rol vive en la propia fila de usuario — para el alcance actual (dos roles fijos) una tabla de permisos granulares sería complejidad sin beneficio real.
Site	name , kind , created_at , updated_at	Es la unidad real de agrupación: un usuario nunca posee un dispositivo directamente, siempre a través de pertenecer a un sitio que lo contiene.
SiteMember	site_id , user_id (par único), role (owner / member)	Relación muchos-a-muchos entre usuarios y sitios. Solo el rol owner puede borrar el sitio — un member puede operar los dispositivos pero no dismantelar el sitio.

Dispositivos y seguridad eléctrica

Entidad	Columnas relevantes	Justificación de diseño
DeviceProfile	site_id (opcional), max_current_a , min_voltage_v / max_voltage_v (opcionales), auto_cutoff_enabled	site_id es opcional: un dispositivo recién fabricado igual necesita un perfil de fábrica con umbrales seguros por defecto.
Device	public_id (único, visible), site_id , profile_id , desired_state / actual_state , is_locked_out , last_seen_at , firmware_version	Separar desired_state de actual_state reconoce que una orden enviada no prueba que el relé haya cambiado físicamente.
DeviceCredential	device_id (único, 1:1), secret_hash , revoked_at	Vive en su propia tabla para que el hash del secreto nunca aparezca por accidente en una respuesta que devuelva datos generales de un dispositivo.

Datos operativos e históricos

Entidad	Columnas relevantes	Justificación de diseño
TelemetryReading	device_id , sequence , voltage , current , power , frequency , power_factor , energy , recorded_at	Tabla append-only. sequence detecta telemetría duplicada o fuera de orden.
Command	device_id , type , payload (JSON), status , acked_at	Cola simple sin necesidad de un message broker externo.
Event	device_id , type , payload (JSON)	El campo payload en JSON permite que cada tipo de evento guarde sus propios datos sin necesitar una columna nueva.
AnomalyAlert	device_id , power , expected_power , detector , acknowledged	Nunca corta la energía por sí sola — es información para que una persona la revise.

Notificaciones, tarifas y predicciones

Entidad	Columnas relevantes	Justificación de diseño
Notification / NotificationPreference	user_id , event_id (opcional), channel , title / body , read_at / channel , enabled	Las preferencias son por canal (in_app , push), con un default distinto para cada uno.
Tariff	site_id , currency , price_per_kwh , valid_from / valid_to	Nunca se sobrescribe una tarifa existente — se cierra y se crea una nueva, para poder prorratear el costo cuando el precio cambia a mitad de período.
Prediction	device_id , generated_at , horizon , projected_kwh / projected_cost , method	El resultado se persiste en vez de recalcularse en cada consulta.

4.3. API expuesta

Todas las rutas viven bajo el prefijo `/api/v1/`

Superficie `/auth`

Método y ruta	Qué hace
POST <code>/auth/register</code>	Crea un usuario nuevo.
POST <code>/auth/login</code>	Valida credenciales y devuelve un JWT firmado.

Superficie `/iot` — el firmware

Método y ruta	Qué hace
POST <code>/iot/telemetry</code>	Recibe una lectura; devuelve el comando pendiente (si hay) y el umbral de seguridad vigente.
POST <code>/iot/heartbeat</code>	Señal de "sigo vivo" cada 30 segundos.
GET <code>/iot/commands/pending</code>	Vía de respaldo: lista los comandos que el dispositivo todavía no confirmó.
POST <code>/iot/commands/{id}/ack</code>	Confirma que un comando se aplicó.
WS <code>/iot/ws</code>	Canal en tiempo real: el backend empuja comandos nuevos apenas se generan.

Superficie `/app` — usuarios finales

Método y ruta	Qué hace
GET / POST <code>/sites</code>	Lista los sitios del usuario / crea uno nuevo (queda como <code>owner</code> automáticamente).
PATCH / DELETE <code>/sites/{id}</code>	Renombra un sitio (cualquier miembro) / lo borra (solo el dueño).
DELETE <code>/sites/{id}/membership</code>	Salida propia de un sitio — nunca de otro usuario. Es lo que usa el botón de desvinculación rápida del acceso de soporte temporal.
POST <code>/sites/{id}/devices</code>	Da de alta un dispositivo nuevo dentro de un sitio propio.
GET <code>/sites/{id}/devices</code>	Lista los dispositivos de un sitio.

POST /devices/claim	Vincula a la cuenta un dispositivo ya existente por su <code>public_id</code> .
POST /devices/{id}/unlink	Desvincula un dispositivo del sitio actual (contraparte de <code>claim</code>) — cualquier miembro del sitio puede hacerlo.
GET /devices/{id} (/telemetry /events /metrics /cost /consumption /prediction)	Detalle, historial y métricas derivadas de un dispositivo.
POST /devices/{id}/switch	Encender/apagar — rechazado si el dispositivo está bloqueado por un evento crítico.
POST /devices/{id}/reactivate	Única vía para limpiar un bloqueo crítico.
GET /notifications	Notificaciones del usuario autenticado.
GET / PATCH /notifications/preferences	Lee y ajusta qué canales de notificación quiere recibir el usuario.
WS /app/ws	Notificaciones y telemetría en vivo de los dispositivos en pantalla.
POST /voice/query opcional	Solo existe si el asistente de voz está activado.

Superficie /admin — administración de la plataforma

Método y ruta	Qué hace
POST / GET / PATCH / DELETE /users/{id}	CRUD completo de cuentas de usuario. El borrado admite <code>?delete_orphaned_sites=true</code> (ver sección 18).
GET /users/{id}/deletion-impact	Vista previa de qué sitios quedarían sin dueño si se borra a ese usuario, antes de confirmar.
POST / GET / PATCH / DELETE /sites/{id}	CRUD completo de sitios, con el correo del dueño real y si el admin que consulta está vinculado (<code>owner_email</code> , <code>admin_is_member</code>).
GET / POST / DELETE /sites/{id}/members/{user_id}	Gestión de membresías de un sitio — incluye el flujo de "vincularse para dar soporte".
POST / GET /sites/{id}/tariffs	Alta y listado de tarifas históricas de un sitio.
POST / GET / PATCH / DELETE /devices/{id}	CRUD completo de dispositivos.
POST /devices/{id}/reassign	Mueve un dispositivo a otro sitio, o lo desvincula.
POST / GET / PATCH / DELETE /profiles/{id}	CRUD completo de perfiles de seguridad.
GET /events / GET /anomalies	Eventos y anomalías de toda la plataforma.
GET /dashboard/(metrics by-site by-profile overview)	Métricas agregadas para el panel de administración.

Respecto al flujo end-to-end de una lectura de telemetría. Cada salto es independiente y con su propia red de seguridad: si la notificación falla porque el usuario no está conectado, la lectura ya quedó guardada y el evento ya quedó registrado. Si el comando pendiente falla por un problema de red del firmware, el dispositivo lo recoge en su próximo poll HTTP de todas formas.

Separación de superficies de API

Superficie	Audiencia	Autenticación	Por qué está separada
/iot/*	El firmware	Credencial propia del dispositivo	Un dispositivo físico no tiene contraseña ni sesión, y no debería poder acceder a nada que no sea sus propios datos.
/app/*	Usuarios finales	JWT de usuario	Su alcance se limita estrictamente a los sitios/dispositivos del propio usuario autenticado.
/admin/*	Administración	JWT con rol admin	Separar el CRUD completo de la superficie pública reduce el riesgo de que un error de autorización exponga funciones administrativas.

4.4. Seguridad: autenticación, cifrado y los dos niveles de corte

Diseñamos dos niveles de protección eléctrica:

Nivel 1. Corte crítico local (firmware): el ESP32 corta la carga por sí mismo si la corriente supera el umbral guardado localmente.

Nivel 2. Motor de reglas del servidor: evalúa cada lectura contra sobrecarga, sobrevoltaje y subvoltaje; puede bloquear el dispositivo (`is_locked_out`) con trazabilidad completa. Un dispositivo bloqueado nunca se reactiva solo requiere una reactivación explícita, registrada con quién la autorizó.

Autenticación: tres mecanismos distintos

Actor	Mecanismo	Detalle
Dispositivo	Credencial propia	Header <code>Authorization: Device <public_id>:<secreto></code> . El secreto se genera una única vez en el servidor y se entrega en texto plano solo en ese momento.
Usuario / admin	JWT	Emitido en <code>/auth/login</code> . El rol viaja dentro del token, firmado.
Autorización por pertenencia	<code>SiteMember</code>	Cada operación valida además que el usuario sea miembro del sitio en particular.

Todo el tráfico entre clientes y backend viaja sobre HTTPS/WSS en el despliegue completo, gracias al certificado TLS que emite Caddy. Tanto la contraseña de un usuario como el secreto de un dispositivo pasan por bcrypt antes de tocar la base de datos el texto plano no se conserva en ningún lugar del sistema después del momento en que se genera.

Una limitación reonocida es que el cliente HTTPS del firmware no valida el certificado del servidor (`WiFiClientSecure::setInsecure()`) aceptable para instalaciones locales/LAN, pero debería resolverse con verificación de certificado pineado antes de un despliegue expuesto a una red no confiable.

4.5. Comunicación en tiempo real: WebSocket sobre REST existente

Se agregó WebSocket no para reemplazar el polling HTTP existente, sino como una mejora de latencia con red de respaldo: si el canal WebSocket está activo, un comando llega en fracciones de segundo; si no lo está, el mismo comando se recoge en el siguiente ciclo de poll. En ningún escenario se pierde un comando. El backend mantiene tres registros de conexiones WebSocket separados (`app/core/ws_manager.py`): notificaciones por usuario, telemetría en vivo para quien está mirando un dispositivo, y el canal de comandos hacia el propio dispositivo.

4.6. Inteligencia del sistema: predicción de consumo y detección de anomalías

Agregamos un asistente IA capaz de hacer una predicción de consumo. Para lograrlo se calcula una tendencia lineal simple para proyectar el consumo del resto del período. Se eligió un modelo simple y explicable a propósito: con el volumen de datos disponible por dispositivo, un modelo más complejo no aportaría precisión real, y sí perdería la propiedad de ser auditable.

Detección de anomalías

- Patrón strategy Detector basado en reglas: z-score sobre el historial reciente — explicable con estadística básica, sin entrenar nada.
- Detector con aprendizaje automático: Isolation Forest (scikit-learn), reentrenado sobre el historial reciente de cada dispositivo.

Ambos implementan la misma interfaz, así que el resto del sistema puede intercambiar uno por otro con un solo punto de configuración.

Sobre el motor de reglas y el detector de anomalías se construyó una capa de notificaciones: cuando pasa un evento relevante, se genera una notificación persistida y, si el usuario tiene una conexión WebSocket activa, se le empuja en el momento.

Las preferencias de notificación ahora se pueden leer: cada usuario ve el estado real de sus canales (`in_app` , activado por defecto; `push` , desactivado por defecto) antes de cambiarlos. Como capa opcional y desacoplada, se integró un asistente de voz (`IA-Assistant/`) que interpreta comandos simples en lenguaje natural y los traduce a las mismas operaciones que ya expone la API de usuario. El backend arranca y funciona completo sin este módulo si está desactivado por configuración.

4.7. El dashboard web (front/)

Construimos una aplicación de una sola página construida en React 19, TypeScript y Vite, que sirve dos audiencias en el mismo build: la persona usuaria final y la administración de la plataforma.

El cliente de la API no se escribe a mano: se genera automáticamente (`openapi-typescript`) a partir del documento OpenAPI real del backend. Si el backend cambia la forma de un dato, cualquier parte del frontend que lo use incorrectamente deja de compilar de inmediato.

El dashboard cubre gestión completa (crear, ver, editar, borrar) de sitios, dispositivos y perfiles de seguridad tanto en la vista de usuario como en la de administración; gestión de membresías y tarifas por sitio; control de encendido/apagado y reactivación; visualización de telemetría, costo, consumo y predicción; y notificaciones en vivo.

El panel de administración necesitaba distinguir, sitio por sitio, de quién es cada uno antes mostraba todos los sitios de la plataforma sin ninguna referencia de propiedad. Se agregó una columna Dueño (el correo del SiteMember con rol owner) y una columna Soporte que marca, con una insignia, si el administrador que está viendo la lista está vinculado a ese sitio en ese momento.

Sobre esa base se construyó un flujo explícito de soporte temporal: un botón "Vincularme temporalmente para dar soporte" agrega al admin como member (nunca como owner nunca se apropia del sitio) de un sitio ajeno, pensado para revisar una anomalía o resolver un problema puntual. La propia vista de "Mis sitios" del usuario refleja este mismo concepto desde el otro lado, con una insignia en cada tarjeta: Dueño si el usuario es el propietario, Soporte temporal si es un admin vinculado por

ese motivo, o Invitado (con el correo del dueño real) si es un miembro normal agregado por otra persona. Se agregó además un botón de salida directa desde esa misma tarjeta (`DELETE /api/v1/app/sites/{id}/membership`), para que un administrador no tenga que volver al panel de administración para dejar de estar vinculado.

Por otro lado, borrar un usuario no tenía ninguna verificación de impacto: si ese usuario era el único dueño de un sitio con dispositivos activos, el sitio quedaba huérfano sin ningún aviso. Se agregó una vista previa (`GET /admin/users/{id}/deletion-impact`) que calcula, antes de confirmar, qué sitios quedarían sin dueño y cuántos dispositivos tiene cada uno, información que se muestra en el propio diálogo de confirmación, con una casilla explícita para borrar también esos sitios (y un aviso de que la acción es irreversible y que los dispositivos de esos sitios quedarán sin sitio, disponibles para vincularse de nuevo más adelante). Un sitio con más de un dueño nunca aparece en esta lista ni se ve afectado, sin importar la casilla.

Las confirmaciones de borrado vivían embebidas dentro de la propia fila de una tabla o tarjeta angosta un patrón que colapsaba visualmente en cuanto el contenido crecía (por ejemplo, el aviso de sitios huérfanos de la sección anterior). Se introdujo un componente de diálogo modal reutilizable, construido sobre Radix UI Dialog (una librería de comportamiento accesible, sin estilos propios, que se integra con la identidad visual ya existente de la plataforma en vez de imponer una propia), y se migraron todas las confirmaciones de borrado y desvinculación tanto en la vista de administración como en la de usuario final a este mismo componente.

Los mensajes de error y éxito de cada acción (crear, editar, borrar, reasignar) vivían como texto embebido junto al control que los disparaba, ocupando espacio de forma permanente hasta que algo más los reemplazaba. Se integró un sistema de avisos flotantes y temporales (toasts), consistente en el dashboard web y en la aplicación móvil, con estilo visual propio de la plataforma (fondo oscuro translúcido, ícono de éxito o error) en vez del estilo por defecto de la librería utilizada.

Durante la integración del sistema de avisos se detectó un problema de fondo, preexistente: el interceptor de peticiones HTTP redirigía a la pantalla de login ante cualquier respuesta 401 del servidor, incluido un intento de login fallido, que naturalmente responde 401. El efecto era que, al fallar el login, el navegador recargaba la página en la que ya se estaba, destruyendo la aplicación antes de que el mensaje de error llegara a mostrarse, sin importar qué mecanismo se usara para mostrarlo. Se corrigió excluyendo explícitamente al propio endpoint de login de esa regla de redirección un 401 ahí se trata como credenciales incorrectas, no como sesión expirada.

Con el sistema operativo en modo oscuro, el texto de los campos de entrada y las listas desplegables de los formularios podía volverse ilegible (texto oscuro sobre fondo oscuro, o viceversa), porque la hoja de estilos le permitía al navegador aplicar su propia paleta de modo oscuro a los controles nativos, sin que el resto de la aplicación tuviera ningún soporte de tema oscuro real. Se corrigió fijando explícitamente el esquema de color a claro para todos los controles de formulario, y se aprovechó el ajuste para además unificar su apariencia visual (fondo translúcido, sombra suave, transición al enfocar) con el resto de la interfaz.

4.8. La aplicación móvil (mobile/)

Comparte la misma capa de funcionalidad de usuario final que front/ , empaquetada como aplicación Android mediante Capacitor. Se eligió Capacitor en lugar de un framework nativo separado porque permite reutilizar la lógica de datos, tipos generados y patrones de UI ya construidos, sobre los que se integraron capacidades verdaderamente nativas: reconocimiento y síntesis de voz del sistema operativo, y notificaciones locales.

4.9. Despliegue: contenedores, proxy inverso y TLS

La plataforma se despliega con Docker Compose, sin ningún paso de docker push ni dependencia de un registro de imágenes: cada máquina construye sus propias imágenes localmente.

La configuración de despliegue completo levanta tres servicios: db (PostgreSQL, sin puertos expuestos al exterior), api (el backend, con migraciones automáticas al arrancar) y proxy (único servicio con puertos publicados). El proxy usa Caddy: reenvía /api/* hacia el backend y sirve el build estático de front/ bajo el mismo origen sin CORS de por medio en producción. Caddy emite un certificado TLS autofirmado propio, adecuado para una instalación local o en red privada.

4.10. Calidad: pruebas automatizadas e integración continua

Cada módulo de lógica de negocio tiene su propio archivo de pruebas (backend-server/tests/), enfocado por responsabilidad: autenticación, motor de reglas de corte y bloqueo, prorrateo de costo, agregación de consumo, predicción y detección de anomalías, entrega de comandos por WebSocket, notificaciones, CRUD

administrativo completo (usuarios, sitios, miembros, tarifas, dispositivos, perfiles), y la vista previa de impacto antes de borrar un usuario. Al último corte registrado, la batería completa suma 110 pruebas, todas en verde, con una cobertura de código de aproximadamente 94%. Un flujo de integración continua corre esta batería automáticamente sobre cada cambio subido al repositorio.

5. Resultados

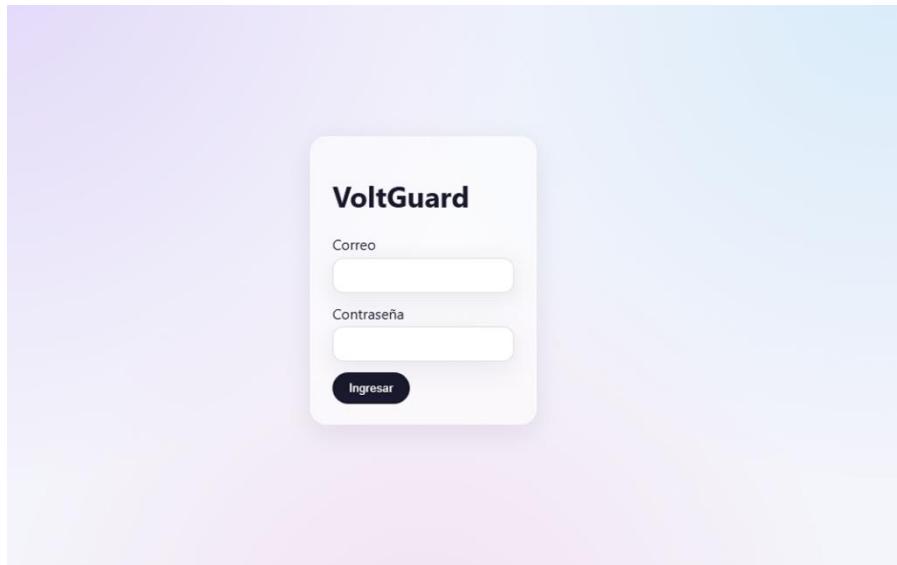
Los resultados recorren los flujos más importantes de la plataforma tal como los vería una persona real, en dos roles distintos:

Rol	Qué puede hacer	Dónde se usa
Administrador de la plataforma	Gestiona usuarios, sitios, dispositivos y perfiles de seguridad de toda la plataforma; puede dar soporte temporal sobre el sitio de otra persona.	Dashboard web (<code>front/</code>), rutas bajo <code>/admin</code>
Usuario final normal	Gestiona únicamente sus propios sitios y dispositivos; enciende, apaga y consulta el consumo de sus equipos.	Dashboard web o aplicación móvil, rutas bajo <code>/sites</code>

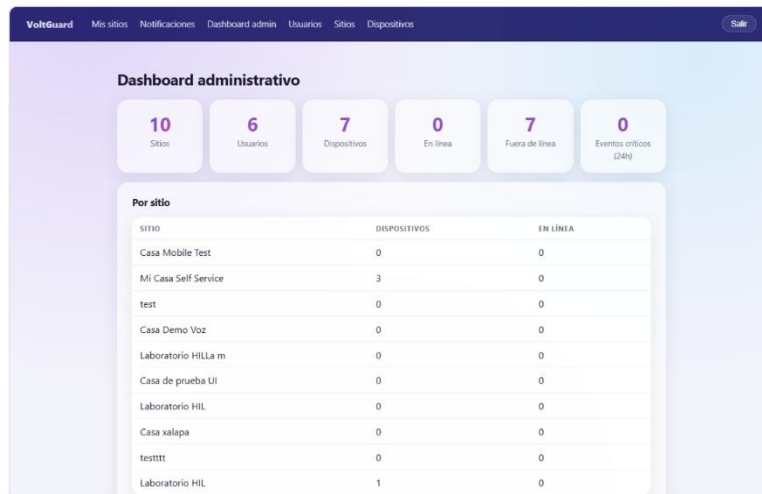
Las capturas de los resultados se tomaron contra el sistema real, con un dispositivo físico conectado (VG-HILREAL-001 , "Foco Laboratorio", Módulo con sensor PZEM-004T sobre un ESP32-C3), por lo que los valores de voltaje, corriente y consumo que aparecen son lecturas reales del hardware, no datos de prueba inventados.

5.1. Casos de uso — Administración de la plataforma

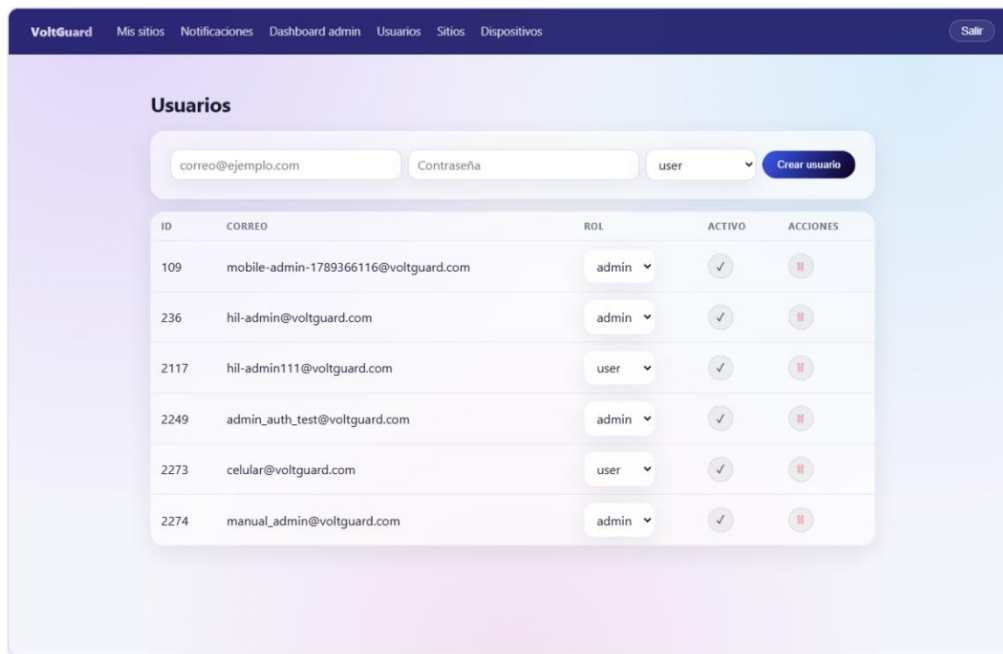
Iniciar sesión: Ir a la pantalla de login e ingresar el correo y la contraseña de una cuenta con rol admin. Pantalla de inicio de sesión, común a usuarios y administradores el rol de la cuenta determina qué secciones aparecen después.



Panel general (dashboard): Al entrar como admin, el menú superior muestra opciones adicionales: Dashboard admin, Usuarios, Sitios y Dispositivos, que no existen para un usuario normal. Esta es la vista del Panel general de administración con métricas agregadas de toda la plataforma.



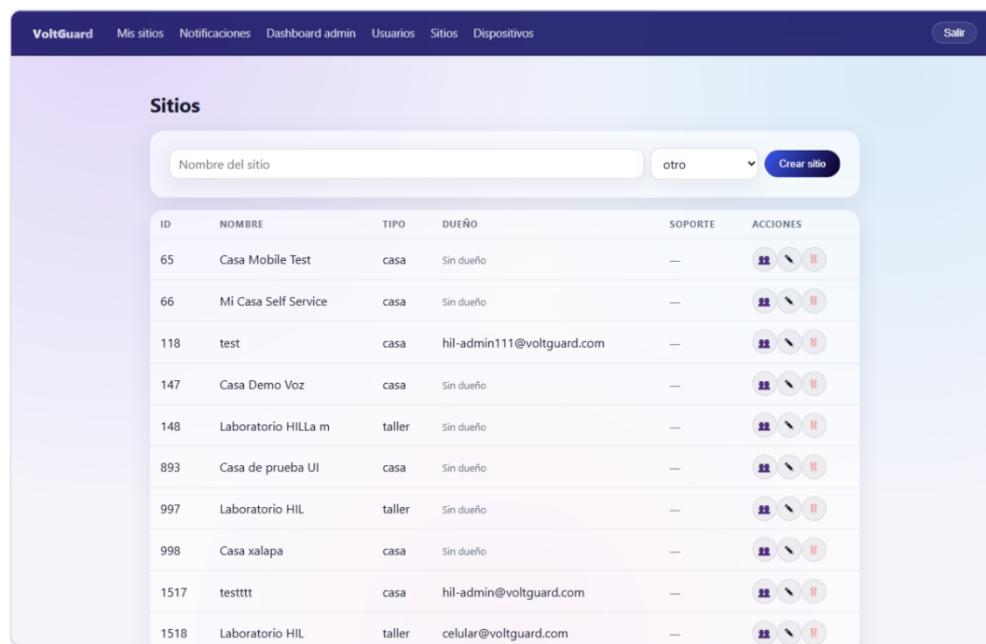
Gestión de usuarios: Para Crear la ruta es: completar correo, contraseña y rol en el formulario superior y presionar "Crear usuario". Editar: el rol y el estado activo/inactivo se cambian directamente desde la tabla. Borrar: el botón de eliminar abre un diálogo de confirmación que, si ese usuario es el único dueño de algún sitio, muestra una advertencia con la lista de sitios que quedarían sin dueño y una casilla opcional para borrarlos también.



The screenshot displays the 'Usuarios' (Users) management page in the VoltGuard application. The page features a dark blue header with navigation links: VoltGuard, Mis sitios, Notificaciones, Dashboard admin, Usuarios, Sitios, and Dispositivos. A 'Salir' (Logout) button is located in the top right corner. Below the header, the title 'Usuarios' is centered. A form for creating a new user is positioned above a table of existing users. The form includes input fields for 'correo@ejemplo.com' and 'Contraseña', a dropdown menu for 'user', and a 'Crear usuario' button. The table below lists six users with columns for ID, CORREO, ROL, ACTIVO, and ACCIONES. Each user's role and active status can be modified directly in the table, and an action button is provided for each user.

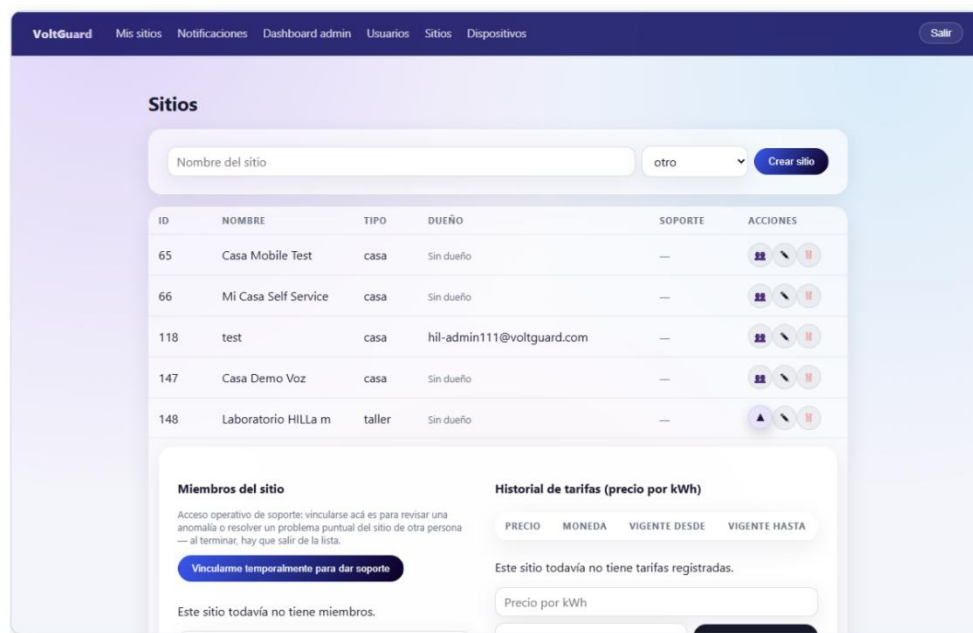
ID	CORREO	ROL	ACTIVO	ACCIONES
109	mobile-admin-1789366116@voltguard.com	admin	✓	[icon]
236	hil-admin@voltguard.com	admin	✓	[icon]
2117	hil-admin111@voltguard.com	user	✓	[icon]
2249	admin_auth_test@voltguard.com	admin	✓	[icon]
2273	celular@voltguard.com	user	✓	[icon]
2274	manual_admin@voltguard.com	admin	✓	[icon]

Gestión de sitios, membresías y tarifas: La tabla de sitios muestra, para cada uno, el Dueño real y si el admin actual está vinculado como soporte. El ícono de personas abre un panel con los miembros del sitio (con un botón directo para vincularse temporalmente) y el historial de tarifas de ese sitio.



ID	NOMBRE	TIPO	DUEÑO	SOPORTE	ACCIONES
65	Casa Mobile Test	casa	Sin dueño	—	  
66	Mi Casa Self Service	casa	Sin dueño	—	  
118	test	casa	hil-admin111@voltguard.com	—	  
147	Casa Demo Voz	casa	Sin dueño	—	  
148	Laboratorio HILLA m	taller	Sin dueño	—	  
893	Casa de prueba UI	casa	Sin dueño	—	  
997	Laboratorio HIL	taller	Sin dueño	—	  
998	Casa xalapa	casa	Sin dueño	—	  
1517	testttt	casa	hil-admin@voltguard.com	—	  
1518	Laboratorio HIL	taller	celular@voltguard.com	—	  

Listado de sitios de toda la plataforma, con dueño real y estado de soporte visibles de un vistazo.



ID	NOMBRE	TIPO	DUEÑO	SOPORTE	ACCIONES
65	Casa Mobile Test	casa	Sin dueño	—	  
66	Mi Casa Self Service	casa	Sin dueño	—	  
118	test	casa	hil-admin111@voltguard.com	—	  
147	Casa Demo Voz	casa	Sin dueño	—	  
148	Laboratorio HILLA m	taller	Sin dueño	—	  

Miembros del sitio

Acceso operativo de soporte: vincularse acá es para revisar una anomalía o resolver un problema puntual del sitio de otra persona — al terminar, hay que salir de la lista.

[Vinculame temporalmente para dar soporte](#)

Este sitio todavía no tiene miembros.

Historial de tarifas (precio por kWh)

PRECIO MONEDA VIGENTE DESDE VIGENTE HASTA

Este sitio todavía no tiene tarifas registradas.

Precio por kWh

Panel expandido de un sitio: miembros (con acceso de soporte temporal) y tarifas.

Gestión de dispositivos y perfiles de seguridad: Desde esta pantalla se dan de alta dispositivos nuevos (queda un secreto que se muestra una única vez), se crean y editan perfiles de seguridad (corriente máxima, rango de voltaje, corte automático) y se reasigna cada dispositivo a un sitio distinto desde el propio listado.

ID	NOMBRE	CORRIENTE MÁX.	VOLTAJE (MÍN/MÁX)	CORTE AUTOMÁTICO	ACCIONES
64	Perfil Smoke Voz	10 A	— / — V	Sí	 
65	Perfil Mobile Test	10 A	— / — V	Sí	 
66	Bomba Jardín (auto)	12 A	— / — V	Sí	 

Perfiles de seguridad y dispositivos de toda la plataforma, con reasignación de sitio en línea.

5.2. Casos de uso — Usuario final

Iniciar sesión: Misma pantalla que en el caso anterior, con una cuenta de rol user . En la aplicación móvil, la experiencia es la misma pantalla adaptada a un formato de app nativa.

VoltGuard

Correo

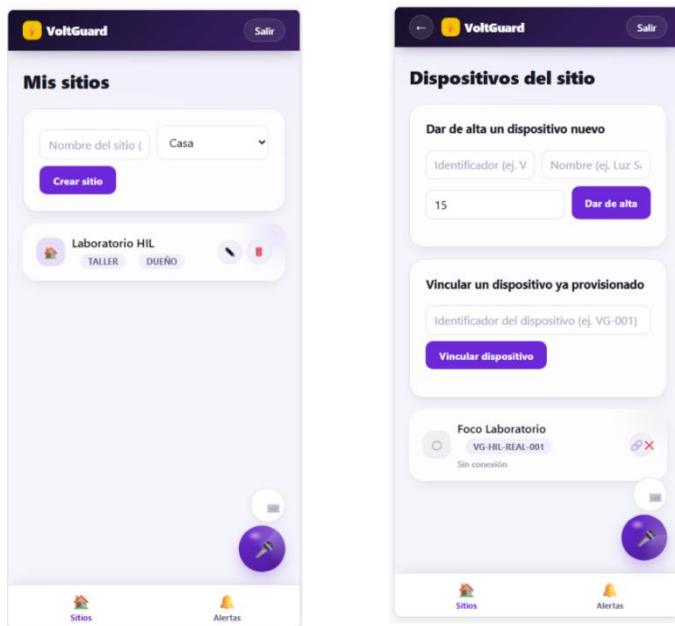
Contraseña

Ingresar

Inicio de sesión en la aplicación móvil.

Mis sitios: Un usuario normal solo ve los sitios donde tiene membresía propia — nunca todos los de la plataforma. Cada tarjeta muestra si es el dueño del sitio o si fue invitado por otra persona.

Dispositivos del sitio: Dentro de un sitio, el usuario puede vincular un dispositivo ya provisionado por su identificador público, dar de alta uno nuevo, o desvincular uno existente. Cada tarjeta de dispositivo muestra su estado actual con un color e ícono congruentes (encendido, apagado, bloqueado, sin conexión).



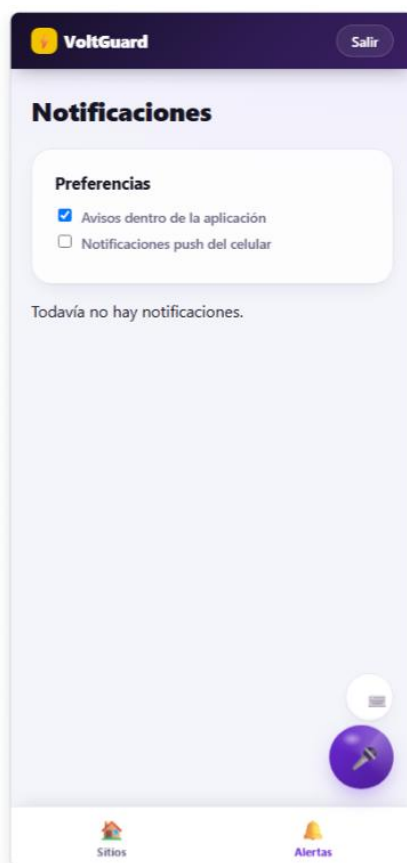
"Mis sitios" y "Dispositivos de sitio" en la app móvil — el sitio "Laboratorio HIL" con el foco real vinculado.

Detalle de un dispositivo: La pantalla de detalle muestra el estado deseado y confirmado del relé, la última lectura del sensor, el costo del período analizado, una gráfica de consumo (con rangos "Hoy", "7 días", "30 días", etc.), la predicción mensual y el historial de eventos — todo calculado sobre datos reales reportados por el dispositivo físico.



Detalle del dispositivo real "Foco Laboratorio": última lectura (125.4 V), costo del período y gráfica de consumo con datos reales

Notificaciones y preferencias: La pantalla de notificaciones lista los avisos recibidos (cortes, anomalías, reactivaciones) y, en la parte superior, permite activar o desactivar cada canal de notificación de forma independiente.



Notificaciones y preferencias por canal en la app móvil.

6. Conclusiones

El desarrollo de VoltGuard permitió evolucionar un módulo electrónico de medición y corte, inicialmente validado de manera independiente, hacia una plataforma IoT integral capaz de supervisar, proteger y administrar cargas eléctricas de forma remota. El resultado no se limita a la lectura de variables eléctricas o al accionamiento de un relevador, sino que integra firmware, comunicaciones, servidor, persistencia de datos, aplicaciones de usuario, administración, análisis de información y mecanismos de despliegue dentro de una misma arquitectura. Esta integración permite que las lecturas generadas por el Módulo lleguen desde un

dispositivo físico real hasta la base de datos y las interfaces de usuario, manteniendo trazabilidad sobre las acciones realizadas y el estado confirmado del dispositivo.

Una de las decisiones más relevantes del proyecto fue mantener al backend como punto central de coordinación. El dispositivo IoT, el dashboard web y la aplicación móvil no se comunican directamente entre ellos; todos consumen contratos definidos por el servidor. Esta separación reduce el acoplamiento entre componentes y permite modificar o sustituir las interfaces de usuario sin alterar el firmware, así como incorporar nuevos clientes que reutilicen las mismas APIs. La división de la plataforma en superficies `/iot`, `/app` y `/admin` permitió, además, separar claramente las necesidades y privilegios del dispositivo físico, del usuario final y del administrador de la plataforma.

Desde el punto de vista de seguridad eléctrica, se concluyó que una función crítica como el corte por sobrecorriente no debe depender de la disponibilidad de Internet ni del servidor. Por esta razón, VoltGuard implementa dos niveles complementarios de protección: un nivel local en el ESP32-C3, capaz de actuar con el umbral almacenado en memoria no volátil, y un segundo nivel en el backend, responsable de evaluar reglas adicionales, registrar eventos, bloquear el dispositivo y conservar la trazabilidad de las acciones. De esta manera, una falla de conectividad puede afectar temporalmente las funciones remotas, pero no elimina la capacidad básica de protección del equipo.

Las pruebas realizadas con hardware real también mostraron la importancia de diseñar los mecanismos de comunicación considerando fallas y latencia. El canal WebSocket permitió reducir el tiempo de entrega de comandos, mientras que el mecanismo HTTP de consulta periódica se conservó como respaldo para evitar que una desconexión del canal en tiempo real implique la pérdida de una orden. Durante las pruebas se detectó además que la reconexión del WebSocket podía bloquear temporalmente el flujo principal del microcontrolador, por lo que este cliente fue trasladado a una tarea independiente de FreeRTOS. Después de esta modificación se registraron tiempos de respuesta en hardware real de aproximadamente 0.27 a 0.53 segundos, demostrando que las decisiones de arquitectura también fueron ajustadas a partir de resultados experimentales y no únicamente de consideraciones teóricas.

Otro resultado importante fue la evolución del modelo de datos desde una relación simple entre usuario y dispositivo hacia una estructura capaz de representar múltiples usuarios, sitios y dispositivos. La incorporación de `Site` y `SiteMember` permite agrupar equipos por casa, laboratorio, taller u otra instalación y establecer relaciones de pertenencia y administración. Del mismo modo, la separación entre `desired_state`

y `actual_state` reconoce que solicitar una acción al dispositivo no equivale a confirmar que dicha acción se ejecutó, lo que mejora la confiabilidad y trazabilidad del control remoto.

La plataforma también demostró que la administración del sistema y la experiencia del usuario final requieren perspectivas diferentes. El usuario puede concentrarse en sus sitios, dispositivos, consumo, estado eléctrico, costos, eventos y control remoto, mientras que el administrador dispone de herramientas globales para gestionar usuarios, sitios, membresías, dispositivos, perfiles de seguridad y métricas agregadas. La incorporación de mecanismos de soporte temporal y la visualización previa del impacto de operaciones destructivas, como la eliminación de usuarios, permitió cubrir situaciones operativas que normalmente aparecen únicamente cuando el sistema comienza a utilizarse de forma real.

La utilización de una misma fuente de contratos para backend, dashboard y aplicación móvil redujo duplicaciones y discrepancias entre clientes. Tanto la interfaz web como la aplicación móvil permiten trabajar con los mismos recursos del servidor, mientras que la generación de tipos a partir del documento OpenAPI facilita detectar incompatibilidades durante el desarrollo. La aplicación móvil, desarrollada sobre Capacitor, permitió además reutilizar gran parte de la lógica existente y añadir capacidades propias del dispositivo móvil.

En cuanto al análisis de datos, se priorizaron soluciones explicables y acordes con el volumen de información disponible. La proyección de consumo utiliza un modelo simple de tendencia, mientras que la detección de anomalías puede realizarse mediante un método estadístico basado en z-score o mediante Isolation Forest. Esta estrategia evita introducir complejidad de aprendizaje automático sin evidencia de que produzca una mejora significativa y mantiene los resultados auditables y comprensibles.

Finalmente, el proyecto alcanzó un nivel de calidad verificable mediante pruebas automatizadas y control de integración continua. Las pruebas documentadas comprenden 110 pruebas con aproximadamente 94 % de cobertura, cubriendo autenticación, reglas de protección, costos, consumo, predicción, anomalías, WebSocket, notificaciones y operaciones administrativas. Esto permite concluir que VoltGuard no fue desarrollado únicamente como una demostración funcional, sino como una plataforma cuya lógica crítica puede verificarse de manera reproducible ante cambios posteriores.

En conjunto, VoltGuard demuestra la viabilidad de combinar instrumentación electrónica, sistemas embebidos, desarrollo backend, aplicaciones multiplataforma y análisis de datos para construir una solución IoT distribuida con separación clara de responsabilidades. La validación con un dispositivo físico real confirma además que la arquitectura propuesta no se limita a datos simulados: el sistema fue capaz de recibir mediciones reales, almacenarlas, presentarlas al usuario y ejecutar acciones sobre la carga conectada.

7. Trabajo futuro

Aunque la plataforma cubre el flujo funcional principal, existen diversas líneas de trabajo que permitirían incrementar su seguridad, robustez, escalabilidad y facilidad de mantenimiento.

En el firmware, una de las siguientes mejoras debe ser incorporar un mecanismo seguro de actualización OTA (Over-The-Air). Actualmente una modificación del firmware requiere acceso directo al dispositivo, mientras que una estrategia OTA firmada permitiría distribuir correcciones y nuevas funcionalidades de manera remota. Este mecanismo deberá incluir validación de integridad, control de versión y recuperación ante una actualización fallida para evitar dejar un dispositivo inoperable. La actualización OTA ya se identifica como una de las líneas de evolución naturales de la plataforma.

También deberá endurecerse la comunicación segura del ESP32. Aunque la infraestructura de producción utiliza HTTPS/WSS, el firmware actual acepta conexiones TLS sin verificar completamente la identidad del servidor mediante `WiFiClientSecure::setInsecure()`. Para un despliegue sobre Internet público será necesario incorporar validación de certificados o certificate/public-key pinning, evitando que un tercero pueda hacerse pasar por el backend aun cuando la comunicación se encuentre cifrada.

Relacionado con lo anterior, resulta conveniente fortalecer progresivamente la seguridad del dispositivo físico mediante mecanismos disponibles en el ESP32-C3, como protección de las credenciales almacenadas en memoria, cifrado de flash y mecanismos de arranque seguro. Esto adquiere especial relevancia porque cada dispositivo posee una identidad y un secreto propios que permiten autenticarse frente al servidor.

El firmware puede evolucionar también hacia un manejo más robusto de eventos y estados críticos. Los eventos generados localmente deberían conservarse de manera persistente cuando el servidor no esté disponible y transmitirse posteriormente al restablecerse la conexión. De esta forma, un corte local producido durante una pérdida de Internet no desaparecería del historial central. Asimismo, puede incorporarse un estado de falla enclavado (fault latch) que impida una reactivación inmediata después de una condición crítica hasta que exista una autorización explícita y se confirme que la condición eléctrica volvió a un rango seguro.

Otra línea de mejora consiste en reforzar la tolerancia a fallos del protocolo dispositivo-servidor. Esto incluye persistencia y reintento de confirmaciones de comandos, identificación inequívoca de ciclos de arranque, protección frente a telemetría duplicada o fuera de orden y almacenamiento temporal de lecturas cuando el backend no esté disponible. Estas medidas permitirían conservar mayor cantidad de información ante interrupciones prolongadas de red y mejorar la consistencia entre el estado físico y el estado almacenado por el servidor.

En el backend deberá incorporarse un mecanismo explícito de rate limiting para la superficie /iot, tanto como protección frente a errores de firmware que produzcan tráfico excesivo como frente a intentos deliberados de abuso. Esta mejora ya fue identificada entre las limitaciones actuales de la plataforma.

La capa de notificaciones también puede ampliarse. El sistema actualmente dispone de notificaciones persistentes y comunicación en tiempo real dentro de las aplicaciones; como evolución, pueden incorporarse servicios reales de notificación push y correo electrónico, así como reglas configurables que permitan al usuario decidir qué eventos justifican cada canal.

En la aplicación móvil, el siguiente paso natural consiste en completar y validar el soporte para iOS, actualmente fuera del alcance documentado. También pueden aprovecharse capacidades propias del dispositivo móvil, como notificaciones push nativas, almacenamiento seguro de credenciales, funcionamiento parcial sin conexión y mecanismos de vinculación de dispositivos mediante códigos QR.

Respecto al análisis inteligente del consumo, será necesario incrementar el volumen y diversidad de datos reales antes de evaluar modelos más complejos. Con históricos suficientemente extensos sería posible comparar formalmente los modelos actuales contra métodos de series temporales, estudiar patrones diferenciados por tipo de carga y estimar intervalos de confianza para las proyecciones. El criterio debe continuar siendo que un incremento de complejidad solo se justifique cuando produzca una mejora medible frente a los métodos simples ya implementados.

También sería conveniente realizar campañas de validación de mayor duración sobre el hardware físico. Además de las pruebas funcionales actuales, estas campañas deberían medir estabilidad durante horas o días de operación continua, recuperación ante pérdidas repetidas de Wi-Fi, reinicios del backend, reconexión de WebSocket, comportamiento del relevador ante múltiples ciclos y coherencia entre el estado físico, el reportado por el firmware y el registrado en la base de datos. Esto permitiría identificar problemas térmicos, eléctricos o de comunicación que no suelen aparecer durante pruebas breves.

Para una eventual evolución hacia instalaciones con un número elevado de dispositivos, deberá evaluarse el comportamiento del sistema bajo carga concurrente. Las pruebas futuras pueden simular cientos o miles de clientes IoT transmitiendo telemetría y manteniendo conexiones WebSocket, midiendo latencia, utilización de CPU, memoria, crecimiento de PostgreSQL y tiempos de consulta. A partir de esos resultados podrá determinarse si la arquitectura monolítica modular actual continúa siendo suficiente o si alguna parte concreta requiere escalamiento independiente; no resulta conveniente introducir esa complejidad antes de que las mediciones lo justifiquen.

Finalmente, como continuación directa del enfoque utilizado durante el desarrollo, se propone mantener una estrategia de validación Hardware-in-the-Loop en la que las nuevas versiones del backend, firmware y aplicaciones deban superar no solamente pruebas unitarias y de integración, sino también una batería reproducible utilizando el dispositivo VoltGuard físico. Esto permitirá comprobar que una modificación que resulte correcta a nivel de software continúa siendo compatible con el comportamiento eléctrico real del sistema.

El trabajo futuro de VoltGuard no requiere replantear la arquitectura desarrollada, sino fortalecer progresivamente cada una de sus capas a partir de evidencia obtenida en operación real. La separación entre hardware, firmware, servidor y clientes permite abordar estas mejoras de manera independiente, conservando los contratos existentes y reduciendo el riesgo de regresiones. El siguiente objetivo debe ser transformar el prototipo funcional validado de extremo a extremo en una plataforma capaz de operar durante períodos prolongados, sobre redes no confiables y con múltiples dispositivos reales, manteniendo como prioridades la seguridad eléctrica, la trazabilidad y la confiabilidad de la información.